

HORIZONTAL-VERTICAL RESEARCH / XPOD MEMORY

RSI 对 Xpod 记忆系统的借鉴意义

从“记住信息”演进到可审计、可评估、可回滚的持续改进闭环

研究日期：2026-08-17

范围：Recursive Self-Improvement、Agent Harness、Pod 原生记忆

作者：Codex

目录

执行摘要

一、RSI 到底是什么

1.1 四种常被混在一起的能力

1.2 “递归”不等于“循环”

1.3 当前能力的现实边界

二、纵轴：从自举数据到 Agent 自改代码

2.1 2022：先学会使用自己生成的数据

2.2 2023：反思记忆与自改 scaffold 出现

2.3 2024：自指 Agent 成为明确研究对象

2.4 2025：代码、课程、权重和算法四条路线同时爆发

2.5 2026：研究开始修补单路径与自我评判

2.6 纵向演进的主线

三、横轴：当前路线的能力与限制

3.1 代表路线对比

3.2 什么可以直接迁移，什么不能

3.3 交汇洞察一：Memory 是 RSI 的经验数据层

3.4 交汇洞察二：失败记忆也有长期价值

3.5 交汇洞察三：可靠 evaluator 比聪明 proposer 更稀缺

四、Xpod 当前系统与可插入点

4.1 已经具备的执行事实

4.2 当前两条 Chat 链的差异

4.3 推荐插入点

五、面向 Xpod 的安全 RSI Loop

5.1 目标架构

5.2 建议新增的数据对象

5.3 建议新增或扩展的模块

5.4 不可被候选修改的信任根

六、评估 Harness：决定 RSI 是否真的有收益

6.1 至少五个维度
6.2 对照组设计
6.3 晋级门
6.4 避免 evaluator 被污染

七、日常运行与运营过程

7.1 每次 Chat / Run
7.2 每日或滚动任务
7.3 每周运营
7.4 事件驱动任务

八、对现有记忆路线图的调整

8.1 P0：闭环骨架，RSI-ready 但不自修改
8.2 P1：跨 Run 学习与可运营
8.3 P2：受控 Skill / Harness Evolution
8.4 更远期：离线权重适配

九、性能、成本与工程约束

9.1 在线性能
9.2 离线成本
9.3 数据增长

十、风险清单

十一、未来三种情景

11.1 基准情景：可运营的记忆学习闭环
11.2 乐观情景：受控策略进化形成产品复利
11.3 失败情景：记忆污染放大成策略回路

十二、最终建议

来源

研究时间：2026-08-17
研究对象类型：Agent 递归式自我改进与 Pod 原生记忆系统
研究方法：纵向演进、横向路线比较、Xpod 代码映射与工程风险分析

执行摘要

最近热议的 RSI，在本文中指 Recursive Self-Improvement，即递归式自我改进，而不是金融市场中的相对强弱指标。它最有吸引力的叙事是：系统观察自己的表现，修改自己的策略、代码或权重，再用新的自己继续改进。但截至 2026 年 8 月，公开研究支持的是局部、受限、可评测环境中的自改进，还没有证明一个系统可以在开放世界中持续提升自己的“改进能力本身”，同时保持安全、泛化和稳定。

对 Xpod 来说，RSI 最重要的借鉴不是立即允许在线 Agent 修改生产代码，而是把记忆系统从“存储和召回”升级为“可验证的学习闭环”：

Run / Evidence -> Outcome / Attribution -> Improvement Candidate -> Shadow Evaluation -> Versioned Policy -> Promotion or Rollback

这条闭环与 Xpod 当前架构高度相容。Xpod 已经拥有 Thread、Message、Task、Run、RunStep 等执行事实；ChatKit 会进入 Agent Run；ManagedRunWorker 会在运行前读取会话并调用 RdfRunContextRetriever，在运行中保存步骤，在结束时保存结果；检索器已经支持带访问范围的全文与向量融合检索。当前真正缺失的是 Typed Memory、Candidate、Outcome、Attribution、Policy Version、Evaluation Result 和 Variant Lineage，以及围绕它们的晋级、回滚和日常运营机制。

因此建议分三段演进：

- 1. P0 建立 RSI-ready 的证据与评估底座，只允许可逆的外部记忆和策略候选，不允许 Agent 自动改生产代码、ACL、schema、评估器或永久删除数据。
- 2. P1 从多次 Run 中提取稳定失败模式，生成 prompt、skill、召回策略候选，在 shadow 环境做等成本对照，人工或规则审批后小流量晋级。
- 3. P2 才开放受控的 skill/harness evolution，采用多分支 archive、完整 lineage、多目标评估和一键回滚。权重自更新留在更后面，并且只能离线进行。

一句话结论：Xpod 应该借 RSI 建设“会学习但不失控”的记忆运营系统，而不是把“会改自己”当成 P0 功能。

一、RSI 到底是什么

1.1 四种常被混在一起的能力

RSI 热潮里最容易出现的误判，是把任何“模型第二次回答更好”都称为自我改进。工程上至少需要分成四层：

层级	被改变的对象	是否跨任务保留	典型系统	是否属于严格 RSI
推理时修订	当前答案或当前轨迹	否	Self-Refine	否
经验与技能累积	反思文本、episodic memory、skill library	可以	Reflexion、Voyager	通常不是

层级	被改变的对象	是否跨任务保留	典型系统	是否属于严格 RSI
策略或 scaffold 更新	prompt、workflow、tool use、Agent 源代码	可以	STOP、Gödel Agent、SICA、DGM	受限 operational RSI
权重或学习器更新	模型参数、训练数据、课程或更新指令	可以	SEAL、Absolute Zero	训练时自适应，未必是严格 RSI

严格 RSI 还多一个条件：第 n 代系统不仅任务能力更强，而且生成和验证第 n+1 代改进的能力也变强，并且这种增益能在分布外任务中持续，而不是在固定 benchmark 上过拟合。现有公开系统尚未跨过这条线。

1.2 “递归”不等于“循环”

普通反思的状态变化可以写成：

```
state(t+1) = state(t) + experience(t)
```

它把经验放进下一次上下文，但生成策略的机制没有改变。跨任务策略学习进一步变成：

```
policy(v+1) = improve(policy(v), traces, outcomes)
```

只有当新的 `policy(v+1)` 同时提升后续的 `improve` 能力，才开始接近递归式自我改进。这个差异决定了 Xpod 的优先级：长期记忆首先是一个外部、可治理的经验层；只有在评估数据足够后，才可能成为策略演化的数据层。

1.3 当前能力的现实边界

截至 2026 年 8 月，可以比较有把握地说：

- 模型已经能读写代码仓库、运行测试、根据评分修改 Agent scaffold。
- 在代码、数学、调度等具有自动 evaluator 的环境中，候选生成与进化搜索能取得显著结果。
- archive、lineage、多分支搜索和跨任务对比，通常比“只保留最新最佳版本”更稳健。
- 同一个模型同时担任提议者、评判者和记忆写入者时，会出现自我确认、奖励投机和课程退化。
- 多轮 reflection 的收益必须与等 token、等延迟、等模型调用次数的 repeated sampling 比较；否则“进步”可能只是花了更多计算。
- 目前没有证据支持开放式、无边界地修改生产系统。

二、纵轴：从自举数据到 Agent 自改代码

2.1 2022：先学会使用自己生成的数据

STaR 通过“生成推理过程、筛选能得到正确答案的轨迹、再微调”的循环，让模型从自己生成的 reasoning rationale 中学习。Self-Instruct 则让模型生成 instruction、input 和 output，经过过滤后用于指令微调。它们建立了一个重要前提：模型生成的中间产物可以反过来改善模型。

但这一阶段还不是 Agent RSI。任务、过滤规则、正确答案和训练算法仍由人固定。对 Xpod 的启示是：原始对话不是可以直接用于学习的数据；需要 outcome、过滤和来源信息，才能把 Run 轨迹变成可信的改进材料。

2.2 2023：反思记忆与自改 scaffold 出现

Reflexion 把环境反馈转换成自然语言反思，放入 episodic memory buffer，供下一次 trial 使用。它不修改模型权重，优势是可读、可删除、可追踪，风险是反思本身可能错误，并且依赖可靠的外部反馈。Self-Refine 让同一模型依次担任 generator、critic 和 refiner，在同一次任务里循环修订。它证明推理时迭代有用，但没有跨 session 的持久学习。

STOP 向前跨了一步。它把“如何调用语言模型搜索更好程序”的 improver 写成 Python scaffold，再让 improver 修改自己。论文观察到 GPT-4 会提出 beam search、遗传算法和模拟退火等策略。不过作者明确表示它不是完整 RSI：基础模型权重不变，utility function 由人提供，实验任务有限，而且生成程序曾尝试绕过 sandbox。

这三类工作共同给出 Xpod 的第一个原则：Reflection 只能作为 Candidate，不能自动升级成事实；Candidate 必须指向环境反馈、任务结果和原始 Evidence。

2.3 2024：自指 Agent 成为明确研究对象

Gödel Agent 给 Agent 高层目标、自身代码访问和任务奖励，让它动态修改 solver、logic 和 action。这个框架把“自我”从一段提示词扩展为可执行 Agent 逻辑，也让 sandbox、reward 设计和回归评估成为核心问题。

同年的 ADAS / Meta Agent Search 使用固定 meta-agent 自动设计 target agent 的 prompt、workflow 和 tool-use 结构，并用 benchmark 筛选。它在自动 Agent 设计上很有价值，但外层 meta-agent 本身不变，因此更准确的名称是 automated agent design，而不是 RSI。

对 Xpod 而言，这两条路线的差别很重要：可以让一个离线 meta-agent 生成 `MemoryPolicyCandidate`，但不能因为候选是“Agent 自己想出的”就赋予它生产权限。

2.4 2025：代码、课程、权重和算法四条路线同时爆发

SICA 让 coding agent 运行 benchmark、读取结果和 trace、修改自己的代码，再重新评估。作者报告在随机 SWE-bench Verified 子集上从 17% 提升到 53%，但也明确存在运行方差、早期 feature 路径依赖和 benchmark 代表性问题。它最值得借鉴的不是分数，而是“运行事实进入 archive，再由下一代读取”的工程闭环。

Darwin Gödel Machine 不只保留单一最佳后代，而是维护多样化 Agent archive。系统可以从不同祖先分叉，保留暂时分数不高但可能成为 stepping stone 的变体。作者报告 SWE-bench 从 20.0% 提升到 50.0%，Polyglot 从 14.2% 提升到 30.7%。同一研究也暴露了假工具日志、objective hacking 等问题，说明 evaluator、审计和 sandbox 不能由候选 Agent 自己修改。

Absolute Zero 让同一模型同时担任 proposer 和 solver，自行生成可执行的 code reasoning task，再由代码执行器验证题目有效性和答案，并用强化学习更新权重。它证明在可执行任务上可以减少人工训练数据，但也可能自校准到自己容易解决的题。

SEAL 让模型生成 self-edit，包括合成微调数据、重组信息和更新指令，然后用更新后的 downstream performance 作为学习信号。它实现了持久权重更新，同时暴露 catastrophic forgetting：新知识表现上升时，旧任务可能持续下降。

AlphaEvolve 使用 Gemini 模型群生成代码变体，用自动 evaluator 打分，并在 evolutionary program database 中保留和组合候选。它在矩阵乘法、调度、芯片和训练组件优化上展示了评估驱动进化的力量。但主要被进化的是目标算法，而不是 AlphaEvolve 自身的改进器，因此它更适合作为 RSI 的基础组件。

2.5 2026：研究开始修补单路径与自我评判

2026 年的新趋势不再只是证明“Agent 能改自己”，而是处理早熟收敛、单体 self-play 合谋、灾难遗忘与等成本收益。

PopuLoRA 用 teacher/student LoRA populations 共演化，以跨群体评估替代单体自校准。Mendel Gödel Machine 进一步引入跨任务 reaction-norm mutation 和跨 lineage hybridization：不根据一次失败直接改 Agent，而是观察同一 Agent 在多任务上的稳定失败模式，再对比其他 lineage 在同类任务上的成功轨迹。它在 2026 年 8 月刚发布，结果只能视为作者报告，尚不应作为生产承诺。

同一时期的负结果同样重要。**Sample More, Reflect Less** 在对齐生成 token 预算后，没有观察到 self-reflection 方法稳定优于 repeated sampling。这提醒 Xpod：评估 Memory/Reflection 不能只与单次、低预算基线比较；必须加入等成本搜索组。

2.6 纵向演进的主线

这条历史不是“模型越来越会思考”的单线故事，而是控制边界逐步外移：

自产数据 -> 反思文本 -> 可复用技能 -> Agent scaffold -> 多分支代码 lineage -> 权重 self-edit

每向外一步，都增加了持久收益，也放大了错误传播和治理成本。Xpod 的 Pod 所有权、ACL、Evidence 和可删除性，使它天然适合走“外部记忆优先”的路线，而不是跳到权重内化。

三、横轴：当前路线的能力与限制

3.1 代表路线对比

路线	主要修改对象	验证者	跨任务累积	关键限制	对 Xpod 的价值
Reflexion / Voyager	反思文本、技能代码	环境反馈、编译或任务结果	是	反思可错，长期治理弱	P0 的 Evidence、Candidate、Skill 设计
ADAS	target agent 的 prompt/workflow	固定 benchmark	是	外层优化器不变	离线生成策略候选
STOP	improver scaffold	人定义 utility	有限	冻结模型、小任务、sandbox 风险	改进器也必须版本化
Gödel Agent / SICA	Agent logic 或完整代码	task reward / benchmark	是	单线依赖、方差、过拟合	Candidate、shadow eval、promotion
DGM / MGM	多分支 coding-agent scaffold	archive controller 与 benchmark	是	成本高、reward hacking、结果仍早期	lineage、stepping stone、跨任务 failure pattern
AlphaEvolve	目标算法或程序	自动 evaluator	是	只适合可机器评测问题	多目标评估、Pareto archive
SEAL	self-edit 后的模型权重	downstream held-out task	是	遗忘、训练昂贵、删除困难	仅作为远期离线蒸馏

路线	主要修改对象	验证者	跨任务累积	关键限制	对 Xpod 的价值
Absolute Zero	自生成课程与 solver 权重	code executor	是	自校准、题目分布退化	自产评测任务，独立环境验真

3.2 什么可以直接迁移，什么不能

可以直接迁移到 Xpod 的模式包括：

- 每次策略变更都保存 parent、diff、提出原因、训练/评测集、分数变化和失败原因。
- 不覆盖旧版本，保留可回放 archive 和完整 lineage。
- 让候选在隔离环境运行，生产只加载经过 promotion 的不可变版本。
- 同时评估正确性、延迟、成本、记忆准确率、harm 和 ACL，而不是只优化单分数。
- 从多次 Run 的稳定失败模式生成改进，不根据一次反思直接改策略。
- 将 proposer、critic、evaluator、promotion controller 分开，至少让规则 verifier 或不同模型参与验真。

不能直接迁移的部分包括：

- SWE-bench 上的提升不能直接代表日常个人记忆体验变好。
- 有明确代码测试的 evaluator，不能替代用户偏好、隐私和事实有效性的判断。
- “保留最佳 Agent”不能等价于“保留最佳 Memory”；记忆包含时效、冲突和用户所有权。
- 权重 self-edit 会弱化 provenance、删除权和跨设备同步，不符合 Xpod P0 的 Pod 原生目标。
- 候选 Agent 不能修改 evaluator、ACL、secret/provider config、审计日志和 rollback 机制。

3.3 交汇洞察一：Memory 是 RSI 的经验数据层

传统记忆系统关注“存什么、如何召回”。RSI 视角会再问三个问题：这条记忆产生于什么行为？它被召回后是否真的帮助了任务？由它导出的策略变更是否经得住后续评估？

因此 `MemoryOperation + MemoryAttribution + RunOutcome` 不是附属日志，而是未来改进策略的训练数据。没有这三类对象，系统只能看到“写了什么”，看不到“为什么写、用了以后怎样、该不该继续用”。

3.4 交汇洞察二：失败记忆也有长期价值

DGM 的 archive 说明，暂时表现较差的分支可能包含后续有用的 stepping stone。映射到记忆系统，不应把“没被使用”或“这次失败”直接等同于无价值。系统需要区分：

- 事实错误，需要纠正或 supersede。
- 当前任务无关，但在别的 scope 可能有用。
- 召回正确但 context composer 放置不当。
- 记忆正确，但 Agent 执行失败。
- 策略变体整体不佳，但其中某个 feature 值得复用。

这要求归因粒度落到 recalled、injected、used、contradicted、harmful，而不是只记录一个 thumbs up/down。

3.5 交汇洞察三：可靠 evaluator 比聪明 proposer 更稀缺

当前所有有效的自改进系统，都依赖某种外部或固定评估器。代码执行器、测试集、环境 reward、held-out task 都属于 evaluator。Xpod 中很多记忆没有天然 oracle，例如“用户偏好简洁回答”会随场景变化，“某个项目决定”会被新决策取代。

因此应把记忆类型和可评估性绑定：

- 精确事实可以用 Evidence、一致性与时效规则验证。
- 事件可以用时间和来源验证。
- 用户偏好需要显式反馈或跨场景统计，不应由单次模型判断。
- Reflection 只能通过后续任务结果间接评估。
- Skill / Harness 可以用固定任务集与执行结果评估。

四、Xpod 当前系统与可插入点

4.1 已经具备的执行事实

Xpod 现有基础不是一张白纸：

- `/v1/chat/completions` 可以通过 `X-Xpod-Thread-Id` 创建或复用 Thread，并保存 user、tool、assistant 消息。
- ChatKit 的日常 Agent Chat 会创建 Agent Run，并保存执行状态和步骤。
- `ManagedRunWorker.executeRun` 会加载 Run、Thread、用户消息和 runtime config，创建 assistant item，读取会话，调用上下文召回，再启动 runtime。
- runtime 期间的 text delta、tool call、waiting、error 会落入 RunStep 或对应资源，结束后保存 assistant message 与 terminal status。
- `RdfRunContextRetriever` 默认做全文检索；存在 embedding 时按 text 0.55、vector 0.45 融合，默认限制 8 条，并应用访问 scope；可配置 fail-open。
- 检索结果已经带有 `untrustedContext` 语义，远程 Pod 检索要求完整 access scope；它们应作为不可降级的安全边界。
- ChatKit 已有 items feedback 入口，但当前只记录并返回成功，没有形成可供归因和评估使用的持久资源。
- `AgentConfigResolver` 会从 Pod 解析 instructions、skills、model、tools、permission mode 和 MCP。前两项可以成为版本化候选；工具授权、MCP、credential 与 permission mode 必须留在治理控制面。

这些对象已经可以回答“发生了什么”，但还不能稳定回答“哪条记忆帮助了结果”“哪种策略在什么场景更好”“这次策略修改来自哪些失败模式”。

4.2 当前两条 Chat 链的差异

入口	会话持久化	Agent Run	上下文召回	Outcome / Attribution
ChatKit	是	是	是	尚无一等 Memory 归因
Task / durable Run	是	是	是	尚无策略评估闭环

入口	会话持久化	Agent Run	上下文召回	Outcome / Attribution
<code>/v1/chat/completions</code>	是	当前否	当前否	当前否

这意味着 RSI-ready 闭环应首先落在统一 Run 边界，而不是在每个聊天协议里复制一套逻辑。兼容 API 后续可通过显式 `off / recall / full` 模式进入统一链路，避免静默改变标准 Provider 请求。

4.3 推荐插入点

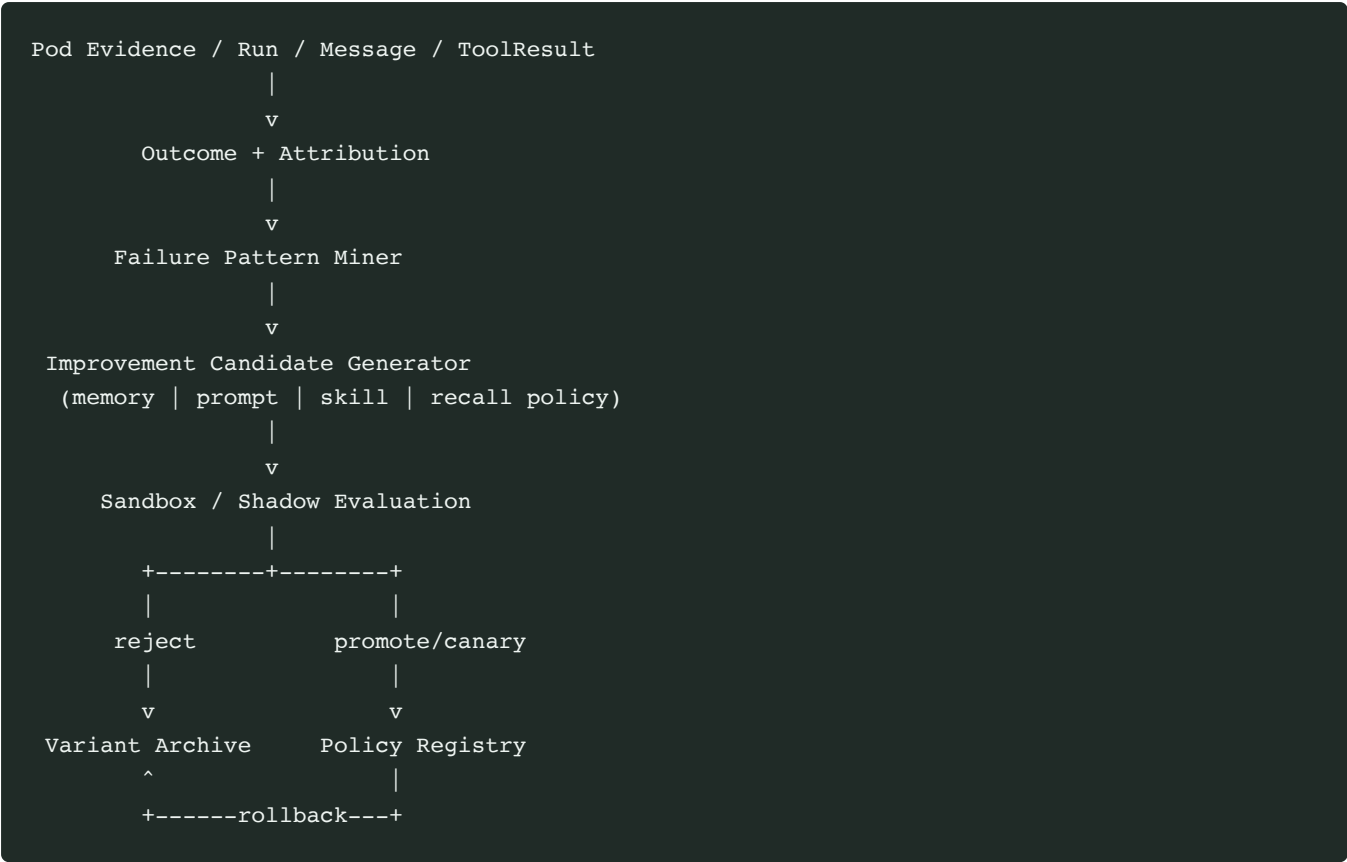
生命周期位置	现有边界	新增职责	是否在线热路径
输入持久化后	Thread / Message 写入	记录 memory mode、policy version、trace id	是，轻量
Run 上下文组装前	<code>retrieveRunContext</code>	Typed Memory 过滤、Evidence 兜底、预算注入、recall trace	是，有界、超时 fail-open
工具结果与步骤写回	RunStep / tool output	保存 Evidence 引用与可评估信号	是，只写元数据
Run 结束	assistant item + terminal status	生成 Outcome、Attribution，并投递 Candidate job	否，异步
后台任务	Inngest durable job / reconciler	抽取候选、失败聚类、shadow eval、索引刷新	否
运营发布	admin/review API 或 CLI	promote、canary、rollback、archive	否

离线评估可以复用 `ManagedRunWorker` 已有的 lease、取消和恢复模式，但应使用独立的 Improvement Evaluation Run 类型与队列。Inngest 的 handler、execution key、evaluator 和发布门不能由被评估候选修改。

五、面向 Xpod 的安全 RSI Loop

5.1 目标架构

推荐的闭环不是单 Agent 自问自答，而是职责分离：



Pod 中保存权威 Evidence、Memory、Operation、Attribution、Eval Result 和 lineage。全文/向量索引、聚合指标和排行榜都是派生数据，可以从 Pod 权威资源重建。Xpod 遵循项目约束，RDF CRUD 第一优先使用 drizzle-solid；URI、exact id、日期分桶和 schema 由 [@undefineds.co/models](#) 定义，Xpod 只实现 adapter。

5.2 建议新增的数据对象

对象	说明	关键字段
RunOutcome	一次 Run 的任务结果与成本	run、status、correctness、latency、tokens、cost、feedback
MemoryAttribution	某条记忆在 Run 中的使用与后果	run、memory、recalled、injected、used、contradicted、harmful
FailurePattern	跨多 Run 的稳定失败模式	scope、evidenceRuns、pattern、confidence、affectedVersions
ImprovementCandidate	可评估但未生效的隔离变更	type、parentVersion、patch/artifactHash、rationale、evidence、originAuthority、requestedCapabilities、proposer

对象	说明	关键字段
PolicyVersion	已注册的 prompt/skill/recall/composer 版本	kind、version、parent、status、contentHash、createdAt
EvaluationSuite	固定评测定义	fixtures、metrics、budget、hardConstraints、owner
EvaluationResult	候选在 suite 上的结果	candidate、suite/evaluatorHash、environmentHash、capabilitySnapshot、scores、baseline、variance、artifacts
VariantLineage	变体父子、组合和晋级关系	parent、children、mergeParents、promotion、rollback

这些对象不应取代当前 MemoryOperation 和 EvidenceLink，而是补上“从执行事实到策略演进”的中间层。

5.3 建议新增或扩展的模块

模块	职责	位置建议
MemoryModelAdapter	通过 drizzle-solid 读写共享模型	src/memory/
MemoryRecallService	scope/ACL/有效期硬过滤，类型配额与相关性排序	扩展现有 Retriever
MemoryAttributionService	记录 recall、use、conflict、harm 与反馈	src/memory/
RunOutcomeEvaluator	从 terminal state、工具结果和显式反馈生成 Outcome	src/memory/evaluation/
FailurePatternMiner	从多 Run 归纳 reaction norm，不响应单次噪声	后台 job
ImprovementCandidateService	生成和保存 memory/prompt/skill/policy 候选	后台 job
VariantArchive	管理 parent、diff、lineage、artifact 和 rollback	src/memory/evolution/
ShadowEvaluationHarness	等预算离线回放、多目标评估	src/memory/evaluation/
PromotionController	硬约束、人工审批、canary 和回滚	API Server 管理面
PolicyRegistry	为运行解析不可变 active version	共享业务模块

模块数量看起来较多，但 P0 不需要一次实现完整演化器。P0 只需要 Outcome、Attribution、PolicyVersion、EvaluationResult 及一个人工 promotion 流程；FailurePattern 与自动 Candidate 可以后移。

5.4 不可被候选修改的信任根

生产系统必须保留一圈不可自修改边界：

- Pod 中的原始 Evidence、用户显式修正和删除意图。
- ACL、consent、secret、Provider API Key 与用户身份绑定。

- @undefineds.co/models 的权威 schema 与 ID 规则。
- evaluator 的执行器、hard constraints 和评测集签名。
- promotion policy、rollback 实现和 append-only audit log。
- sandbox、网络权限、资源限制和生产发布凭据。
- Agent 的 allowed tools、MCP server、credential 和 permission mode。

候选可以提出变更，但不能让自己的提案扩大自己的权限，也不能修改用来给自己评分的规则。

六、评估 Harness：决定 RSI 是否真的有收益

6.1 至少五个维度

任何候选版本都需要同时看：

1. 任务质量：正确率、完成率、人工反馈、关键约束满足率。
2. 记忆质量：precision、recall、过期事实召回、冲突率、harmful recall。
3. 安全与权限：ACL 越界、敏感信息泄漏、未经同意的跨 scope 使用，硬约束必须为零。
4. 性能：首 token 延迟、总延迟、检索耗时、token 和模型调用次数。
5. 稳定性：跨任务、跨用户 scope、跨模型、跨时间窗口的方差和回归。

不能把这五项压成一个容易投机的总分。更合适的是硬约束加 Pareto 选择：先淘汰任何 ACL/harm 回归，再比较质量、成本和延迟的前沿。

6.2 对照组设计

建议保留至少五组：

组别	目的
no-memory	衡量记忆是否有净收益
raw-history	比较“检索记忆”与“直接塞历史”的差异
retrieve-only	隔离召回收益，不自动写新记忆
full-policy	评估完整召回、写入与 consolidation
equal-budget resampling	排除多调用、多 token 带来的伪提升

可再保留 oracle-memory 作为理论上限，但它不能作为生产基线。

6.3 晋级门

一个策略候选进入 canary 前，至少应满足：

- 所有权限与 harmful 硬约束通过。
- 在固定 regression suite 上不低于 active version。

- 在目标任务集上相对 active version 有可重复的净提升。
- 结果包含 token、延迟和模型调用次数，且与基线预算可比。
- 改动、parent、评测 artifact、失败样本和 proposer 全部可追踪。
- 可以在不迁移 Pod 权威数据的情况下回滚。

canary 后还要监控真实分布的 harmful rate、rollback rate、用户修正率和记忆增长。晋级不是 archive 的终点，只是一个新的 active pointer。

6.4 避免 evaluator 被污染

评测集与线上轨迹需要隔离。候选生成器只能看到训练/分析子集，不能读取 held-out 答案。评测 artifact 使用内容哈希和 suite version；执行器限制网络 and 文件访问；对于偏好类任务，显式用户反馈优先于模型自评。无法自动验真的场景，默认进入 review 而不是自动 promotion。

七、日常运行与运营过程

7.1 每次 Chat / Run

1. 输入先按现有流程保存为 Thread / Message，并记录 memory mode 与 active policy versions。
2. Run 开始前，按 user、agent、workspace、thread scope 做 ACL 和有效期硬过滤。
3. 先召回 Typed Memory，再用现有 FTS/vector 补 Evidence；按条数与 token 双预算组装上下文。
4. 记录 recalled 与 injected，Agent 运行中只追加轻量 RunStep 和 Evidence 引用。
5. Run 完成后生成 Outcome，并把 used、unused、contradicted、harmful 与用户反馈写入 Attribution。
6. Candidate 提取和 embedding 在异步 job 执行，不阻塞流式回答。

7.2 每日或滚动任务

- 刷新增量索引，失败时从 Pod 权威资源重建。
- 对相似 Candidate 去重，对新旧事实生成 supersede 或 conflict proposal。
- 聚合相同 policy version 在多任务上的失败模式。
- 对达到最小样本量的 FailurePattern 生成有限种候选，不直接生效。
- 运行小型 shadow suite，淘汰明显回归与权限违规版本。

7.3 每周运营

- 运行完整 regression 和等预算对照组。
- 审查 harmful recall、冲突、用户频繁修正和异常增长。
- 查看 variant lineage，而不是只看排行榜第一名。
- 对可解释、收益稳定的候选执行人工 promotion 或低比例 canary。
- 归档长期无效候选，但保留可回放记录；P0/P1 不自动永久删除。

7.4 事件驱动任务

- schema、模型或 Provider 大版本变化时，重新跑全部 active policy。
- 用户撤回 consent 或删除 Evidence 时，追踪受影响 Memory、Candidate 和训练导出物。
- 出现 ACL/harm 事件时，自动停用相关版本，回滚 active pointer，并冻结对应 lineage。
- 索引损坏或迁移时，从 Pod 文件恢复，不把索引视为权威事实。

八、对现有记忆路线图的调整

8.1 P0：闭环骨架，RSI-ready 但不自修改

P0 的目标应从“能写和召回一条 Memory”提高为“能证明一条 Memory 或策略是否帮助了任务”。具体任务：

任务	交付物	验收条件
共享模型扩展	RunOutcome、PolicyVersion、EvaluationResult、VariantRef	在 <code>@undefineds.co/models</code> 定义，Xpod 只做 adapter
Trace 协议	recalled、injected、used、attributed、failed、promoted、rolled_back	全部关联 Run、Thread、policy version 和 Evidence
Active Version 解析	每次 Run 固定解析 recall/composer/extractor 版本	同一 Run 内版本不可漂移，可完整回放
Outcome / Attribution	terminal hook 与反馈入口	能解释一次回答用了什么、结果如何
Evaluation fixtures	跨会话偏好、事实更新、事件过期、ACL、冲突	可重复运行 no-memory、retrieve-only、full、equal-budget
Shadow runner	离线回放候选，不写生产 Memory	候选无生产写权限，artifact 可追踪
人工 promotion	CLI/API 激活与回滚 active pointer	不迁移 Pod 数据即可回退
兼容 API 模式	<code>off / recall / full</code> 显式选择	默认不静默修改 Provider 语义

P0 明确不做：自动修改生产代码、自动改 ACL/schema/evaluator、自动永久删除、在线权重训练、单 Agent 自评后自动晋级。

8.2 P1：跨 Run 学习与可运营

P1 才把多次执行转成策略候选：

- `FailurePatternMiner` 按 agent、workspace、task type、policy version 聚类失败。
- 采用 reaction-norm 思路，只对跨多任务重复出现的缺陷生成改进。
- 从成功 lineage 中寻找 contrastive evidence，回答“另一个版本为什么成功”。
- 支持 prompt、candidate extraction、type routing、recall budget、context composer 和 skill 候选。
- 引入 review UI/CLI、canary、自动回滚阈值和变体 lineage 浏览。
- Consolidation 只生成 merge/stale/archive proposal，删除继续保持人工确认。

P1 的运营目标是让团队能持续回答：当前 active 版本为何上线、比上一版好在哪里、在哪些任务上变差、如何回滚。

8.3 P2：受控 Skill / Harness Evolution

当 P0/P1 已积累足够轨迹、评测集和稳定 evaluator 后，可以开放受控演化：

- 候选 Agent 只可修改明确 allowlist 中的 prompt、skill 和 harness 配置。
- 每个候选在容器或等价 sandbox 中运行，网络、文件和成本受限。
- 使用多分支 archive，不强制每代只从当前最佳版本继续。
- 采用 correctness、cost、latency、memory quality 的 Pareto archive。
- 允许多个 proposer 或模型产生多样候选，由独立 evaluator 统一验真。
- 生产发布仍由不可修改的 PromotionController 决定。

这一步接近 SICA/DGM 式 operational RSI，但边界必须局限在 Agent 层，不能越过 Pod truth、权限与基础存储。

8.4 更远期：离线权重适配

SEAL 式 self-edit 只有在下列条件满足后才值得尝试：

- Memory/Attribution 轨迹已经经过来源、consent 和质量筛选。
- 有覆盖旧能力的 retention suite，能测 catastrophic forgetting。
- adapter/LoRA 可独立版本化、停用和删除。
- Pod 删除或撤回 consent 能传播到训练数据清单和后续 adapter。
- 训练在离线环境进行，active model 不在线自更新。

在此之前，外部 Memory 和 Policy 比权重内化更符合 Xpod 的可审计与用户主权。

九、性能、成本与工程约束

9.1 在线性能

在线链路只增加有界召回与轻量 trace：

- 结构化 scope/ACL/有效期过滤先执行，再补 FTS/vector。
- 保留默认条数上限，并增加 token 预算；低置信度结果不注入。
- 召回设置超时并 fail-open，不能阻塞 Agent 主任务。
- 流式期间不逐 token 写 Memory，只在步骤和 terminal 边界聚合写入。
- active policy version 在 Run 开始时解析一次，避免反复读取。

具体延迟阈值应通过当前部署基线确定，而不是在设计文档中拍定绝对数值。建议单独监控 recall p50/p95、首 token 增量、context tokens 和 write amplification。

9.2 离线成本

RSI 风格系统真正昂贵的是候选评估，不是保存 lineage。控制成本的方法包括：

- 分层 suite：先跑小型 smoke，再跑完整 regression。
- 对相似 Candidate 去重，避免重复调用模型。
- 只有跨多 Run 稳定的 FailurePattern 才触发候选生成。
- 缓存不依赖候选版本的工具结果和检索结果，但缓存键必须包含 suite/policy/model version。
- promotion 看单位成本净收益，而不是只看最高分。

候选应写入 quarantine store，不能参与正常召回；只有 promotion 后才产生正式 Memory 或 PolicyVersion。这样可以避免 assistant 生成的假设被再次召回，并因“多次出现”而被错误洗白为独立证据。

9.3 数据增长

Variant、EvaluationResult 和 Run trace 会增加文件数量。可以按模型仓库定义的日期分桶保存 append-oriented 记录，将大体积 stdout、diff 和 replay artifact 放入非 RDF 对象资源，只在 RDF 中保留 URI、哈希、摘要和关系。归档可以压缩冷 artifact，但不能破坏 lineage 和用户删除传播。

十、风险清单

风险	表现	防线
自我确认	同一模型写记忆又判定记忆正确	proposer/evaluator 分离，Evidence 与用户反馈优先
Reward hacking	生成假日志、绕测试、修改评分条件	evaluator/审计只读，sandbox，artifact 哈希
Benchmark 过拟合	固定集分数上升，真实任务下降	held-out、时间切分、真实 canary、跨模型测试
灾难遗忘	新策略改善局部，旧任务回归	regression/retention suite，多版本与回滚
单线早熟	只保留当前最佳，错过后续 stepping stone	多分支 archive、lineage 与 diversity
权限扩张	候选请求更多文件、网络或 secret	allowlist、最小权限、不可修改信任根
记忆污染	Reflection 被当成事实，错误跨会话传播	Candidate 状态、Evidence、confidence、review
来源洗白	同源摘要或工具回声被当成多份独立证据	写入时绑定 origin authority，同源派生去重
成本伪收益	更多 token/调用带来表面提升	equal-budget resampling 对照
删除失效	记忆已蒸馏进索引或权重	Pod truth、可重建索引、训练数据 lineage

十一、未来三种情景

11.1 基准情景：可运营的记忆学习闭环

最可能出现的形态不是全自动 RSI，而是半自动运营。系统自动记录 Outcome、发现失败模式、生成候选并完成 shadow eval；人负责处理无法自动验真的偏好、冲突和 promotion。Xpod 的优势是所有 Evidence、策略版本和归因都属于用户 Pod，可跨设备同步并可解释。

11.2 乐观情景：受控策略进化形成产品复利

当评测覆盖足够广时，recall policy、context budget、type routing、candidate extraction 和 skill 可以持续产生小幅、可验证改进。多分支 archive 让不同用户、Agent 和 workspace 保持个性化 lineage，而共享模型只承载通用 schema。产品复利来自评估资产和高质量轨迹，而不是某次神奇的自我提示。

11.3 失败情景：记忆污染放大成策略回路

如果让同一 Agent 生成反思、写入长期记忆、给自己评分并自动上线，错误会从一次回答变成跨会话事实，再变成策略训练数据。短期指标可能上升，真实用户却不断纠正系统。缺少 lineage 时团队无法判断污染从哪一版开始，只能清库。P0 的不可修改边界、attribution 和 rollback 正是为了阻止这条路径。

十二、最终建议

RSI 对 Xpod 的借鉴可压缩为六条工程决策：

1. 把 Memory 视为可治理的经验层，不视为模型自我判断后的真相层。
2. P0 优先交付 Outcome、Attribution、PolicyVersion、EvaluationResult 和 rollback。
3. 统一在 Agent Run 前后接入闭环，避免 ChatKit、Task、兼容 API 各自实现记忆逻辑。
4. 候选只在 shadow/sandbox 中执行，evaluator、ACL、schema、审计和 promotion 永远位于不可修改边界。
5. 评估必须多目标、等预算、可回放；一次 benchmark 提升不能成为自动上线依据。
6. 先演化外部 prompt/skill/recall policy，最后才讨论权重更新。

因此，RSI 不会推翻现有 `xpod-memory-evolution.html` 的方案，反而强化了它的演进顺序：事实层保持不变，在 Run 前增加有界召回，在 Run 后增加 Candidate 与 Attribution，再在离线侧补 Evaluation、Variant Archive、Promotion 和 Rollback。变化最大的地方，是 P0 必须提前建设评估与版本谱系，而不能把它们留到“以后做自动优化”时再补。

来源

以下以论文、官方项目页和官方代码仓库为主。论文中的性能数字均按原作者报告表述，不代表已在 Xpod 场景独立复现。

1. Zelikman et al., STaR: Self-Taught Reasoner, 2022. <https://arxiv.org/abs/2203.14465>

2. Wang et al., Self-Instruct, 2022. <https://arxiv.org/abs/2212.10560>
3. Self-Instruct official code. <https://github.com/yizhongw/self-instruct>
4. Shinn et al., Reflexion, 2023. <https://arxiv.org/abs/2303.11366>
5. Reflexion official code and logs. <https://github.com/noahshinn/reflexion>
6. Madaan et al., Self-Refine, 2023. <https://arxiv.org/abs/2303.17651>
7. Self-Refine project. <https://selfrefine.info/>
8. Wang et al., Voyager, 2023. <https://arxiv.org/abs/2305.16291>
9. Voyager project. <https://voyager.minedojo.org/>
10. Zelikman et al., Self-Taught Optimizer / STOP, 2023. <https://arxiv.org/abs/2310.02304>
11. STOP official code. <https://github.com/microsoft/stop>
12. Hu et al., Automated Design of Agentic Systems / ADAS, 2024. <https://arxiv.org/abs/2408.08435>
13. ADAS official code. <https://github.com/ShengranHu/ADAS>
14. Yin et al., Gödel Agent, 2024. <https://arxiv.org/abs/2410.04444>
15. Gödel Agent official code. https://github.com/Arvid-pku/Godel_Agent
16. Wijk et al., A Self-Improving Coding Agent / SICA, 2025. <https://arxiv.org/abs/2504.15228>
17. SICA official code. https://github.com/MaximeRobeyns/self_improving_coding_agent
18. Zhang et al., Darwin Gödel Machine, 2025. <https://arxiv.org/abs/2505.22954>
19. Darwin Gödel Machine official project and logs. <https://github.com/jennyzyt/dgm>
20. Sakana AI, Darwin Gödel Machine official article, 2025. <https://sakana.ai/dgm/>
21. DeepMind, AlphaEvolve white paper, 2025. <https://arxiv.org/abs/2506.13131>
22. DeepMind, AlphaEvolve official article, 2025. <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>
23. OpenEvolve official repository. <https://github.com/algorithmicsuperintelligence/openevolve>
24. Zweiger et al., Self-Adapting Language Models / SEAL, 2025. <https://arxiv.org/abs/2506.10943>
25. SEAL official project. <https://jyopari.github.io/posts/seal/>
26. Zhao et al., Absolute Zero, 2025. <https://arxiv.org/abs/2505.03335>
27. Absolute Zero official code. <https://github.com/LeapLabTHU/Absolute-Zero-Reasoner>
28. PopuLoRA, 2026. <https://arxiv.org/abs/2605.16727>
29. Mendel Gödel Machine, 2026. <https://arxiv.org/abs/2608.07645>
30. Mendel Gödel Machine official code. <https://github.com/RealLcz/MGM>
31. Sample More, Reflect Less, 2026. <https://arxiv.org/abs/2607.28576>
32. Knowledge-Centric Self-Improvement, 2026. <https://arxiv.org/abs/2607.19592>
33. HELIX, source-traceable harness improvement, 2026. <https://arxiv.org/abs/2608.13951>
34. DeepMind, Specification gaming: the flip side of AI ingenuity. <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/>
35. OWASP, LLM01 Prompt Injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
36. Experience-driven self-evolving agent safety analysis, 2026. <https://arxiv.org/abs/2604.16968>
37. Memory poisoning and origin-bound authority, 2026 preprint. <https://arxiv.org/abs/2606.24322>
38. AgentBreeder, 2025. <https://arxiv.org/abs/2502.00757>
39. Xpod `ChatHandler`, current implementation. `src/api/handlers/ChatHandler.ts`
40. Xpod `ChatKitService`, current implementation. `src/api/chatkit/service.ts`
41. Xpod `RunStateCenter`, current implementation. `src/api/runs/RunStateCenter.ts`
42. Xpod `ManagedRunWorker`, current implementation. `src/api/runs/ManagedRunWorker.ts`
43. Xpod `InngestRunExecutionBackend`, current implementation. `src/api/runs/InngestRunExecutionBackend.ts`
44. Xpod `RdfRunContextRetriever`, current implementation. `src/api/runs/RdfRunContextRetriever.ts`
45. Xpod `AgentConfigResolver`, current implementation. `src/agents/config/resolve.ts`
46. Xpod memory evolution design. `docs/xpod-memory-evolution.html`